

Republic of Cameroon
Peace-Work-Fatherland
Ministry of Higher Education
University of Buea
**Faculty of Engineering and
Technology**
**Department of Computer
Engineering**



République du Cameroun
Paix-Travail-Patrie
**Ministère de l'enseignement
supérieures**
Université de Buea
**Faculty of Engineering and
Technology**
**Department of Computer
Engineering**

Software Design Document for A Car Fault Diagnostic Mobile Application

Course Instructor : Dr Nkemeni Valerie

Group Members

- | | |
|--|-----------------|
| 1- Angi Atangwa Valerine | FE22A148 |
| 2- Av'Nyehbeuhkonh Ezekiel Hosana | FE22A160 |
| 3- Beng Dwe Meinoff | FE22A173 |
| 4- Jong Turkson Junior | FE22A228 |
| 5- Nyuyesemo Calglain | FE22A287 |

Table of Contents

ABSTRACT	4
I- INTRODUCTION	5
I.1 PURPOSE OF THE DOCUMENT	5
I.2 SCOPE OF THE PROJECT	5
I.3 OBJECTIVE	5
II- SYSTEM OVERVIEW	6
II.1- Problem statement and proposed solutions	6
II.2- System core Features and Purpose	6
II.3- Technology Used	7
III- SYSTEM ARCHITECTURAL DESIGN	8
IV- SYSTEM MODELS AND DIAGRAMS	9
IV.1 Context Diagram	9
IV.2 Use case Diagram	11
IV.3 Class diagram	13
IV.4 Sequence diagram	15
IV.5 Dataflow diagram	20
IV.6 Deployment diagram	22
V- DATA DESIGN	24
VI- COMPONENT DESIGN	25
VII- CONCLUSION	25

ABSTRACT

This software design document (SDD) presents a comprehensive overview of the car diagnostic app, a mobile application aimed at assisting vehicle owners in identifying and resolving car issues using automated diagnostics. The document outlines the system's architecture, core functionalities, and interaction models through key system model such as usecase, class, data flow, sequence and deployment diagram.

The SDD ensures clarity of system understanding and design providing a blue print for the implementation, testing, maintenance, efficiency and effectiveness.

I- INTRODUCTION

This document outlines the design and architecture of the car fault diagnostic system, a system aimed at helping car owners easily diagnose their cars by scanning dashboard warning lights and recognition of sound engine. This document provides a detailed description of the system's components, interfaces and data flows. It serves as a guide for developers, testers, and stakeholders to understand the system's design and implementation. The SDD aims to ensure that the system meets the requirements and specifications outlined in the SRS document.

I.1 PURPOSE OF THE DOCUMENT

This System Design Document provides a detailed architectural and design view of the Car Fault Diagnostics App. It aims to guide developers, testers, and stakeholders by illustrating how the system will be built, interact, and deployed. This document serves as a reference guide for the development team, stakeholders, and other interested parties, ensuring that the system is built to meet the specified requirements and functionality.

I.2 SCOPE OF THE PROJECT

The Car Fault Diagnostic App provides real-time monitoring and fault detection capabilities by connecting to a vehicle's OBD-II port using Bluetooth or Wi-Fi. It interprets diagnostic trouble codes (DTCs), displays readable fault descriptions, suggests possible causes, and recommends solutions or next steps. The app aims to empower users with actionable insights without requiring advanced automotive knowledge. Features include fault history logs, live engine parameter viewing, maintenance reminders, and a user-friendly dashboard.

I.3 OBJECTIVE

General Objective

- To provide the system architectural design, providing system models and design.

Specific Objectives

- Provide the architecture used by the system
- Draw system diagrams such as;
 - **Context Diagram**

- **Data flow diagram**
- **Use Case diagram**
- **Class diagram**
- **Sequence diagram and**
- **Deployment diagram**

II- SYSTEM OVERVIEW

II.1- Problem statement and proposed solutions

a) Problem Statement

Car owners often struggle to interpret dashboard warning lights and engine sounds, leading to delayed maintenance, unnecessary mechanic visits, and inflated repair costs. Many rely on mechanics even for minor issues due to a lack of accessible, user-friendly diagnostic tools. Existing solutions either require technical expertise, lack multilingual support, or fail to integrate critical features like sound analysis and offline functionality.

b) Proposed Solution

A mobile application that empowers non-technical users to:

- **Diagnose Issues:** Scan dashboard lights/signs and analyze engine sounds via smartphone sensors.
- **Receive Guidance:** Get explanations of warnings, repair tutorials, and maintenance reminders.
- **Connect with Experts:** Locate nearby mechanics/garages and request professional advice.
- **Work Offline:** Access core features (tutorials, diagnostic history) without internet.

II.2- System core Features and Purpose

a) Core Features:

- Dashboard light/sign recognition via camera.
- Engine sound analysis using ML models.
- Multilingual support (English, French, local languages).
- Maintenance logs and reminders.

- Explanation of warnings, repair tutorials, and maintenance reminders.
- Geolocation-based mechanic/garage suggestions.
- Offline functionality for critical tasks.

Users: Car owners, drivers, mechanics (secondary stakeholders).

b) Purpose:

- Reduce dependency on mechanics for minor issues.
- Lower maintenance costs and extend vehicle lifespan.
- Educate users about car diagnostics through tutorials and expert advice.
- Provide a scalable, intuitive tool compatible with Android and iOS devices.

II.3- Technology Used

Layer	Technology/Tools	Purpose
Frontend	Flutter (Dart)	Cross-platform UI development for Android and iOS users.
Backend	Firebase (Authentication, Firestore, Cloud Functions)	User authentication, real-time data storage, and serverless backend logic.
Machine Learning	TensorFlow Lite, PyTorch (for model training)	On-device sound recognition (engine noises) and dashboard light detection.
Database	Firestore (NoSQL), Hive/SQFlite (local storage)	Cloud-based and offline storage for diagnostic history and user profiles.

APIs	Node.js + Express (REST API), API's for image and sound recognition for dashboard and engine sound analysis.	Integration with third-party services (e.g, mechanics' databases, maps).
Geolocation	Google Maps API, Flutter Geolocator Plugin	Locate nearby garages and mechanics based on user's location.
Image Processing	OpenCV, TensorFlow (for image recognition)	Analyze dashboard lights/signs via camera input.
Offline Functionality	Hive, SQFlite	Enable app usage without internet (e.g., cached tutorials, basic features).
State Management	Provider/Riverpod	Manage app state efficiently across screens.
CI/CD	GitHub Actions, Codemagic	Automated testing and deployment for Android and iOS.

III- SYSTEM ARCHITECTURAL DESIGN

System architecture is an essential part of the systems. It determines how components will be arranged and how the app will work. For our system, we decided to use the multi-layered architecture because it gives room to easy maintenance and allows easy structuring and management of the various app components. Alongside with the multi layered architecture we went forward to choose the architectural design pattern we ought to use, for the system, car fault diagnostic app we ought to use the MVVM (Model view view model) architectural design pattern.

We choosed this pattern beacause of the following reason,

- **Data binding:** it enables easy dta binding which is useful for displaying complex diagnostic data.
- **Separation of concerns(features):** MVVM enables clear separation of concerns of concerns , these concerns include data management, business logic, user interface and security.
- **Flexibility:** MVVM allows for easier integration with various data sources and services.

IV- SYSTEM MODELS AND DIAGRAMS

To effectively design and develop car fault diagnostic app, it is essential to create system models and diagrams that illustrate the app's architecture, components and interactions. These visual representations enable the development team to identify potential issues and ensure that the aoo meets the required functionality and performance standards. This section presents the system models and diagrams for the app including context diagram, class diagram, sequence diagram, data flow diagram, use case diagram and deployment diagram. The app system models demonstrate the relationships between components, data flows and interactions. The different diagrams are outlined below;

IV.1 Context Diagram

A **context diagram** is a high-level, graphical representation of a system that shows:

- **The system as a single process (or entity)**
- **Its interactions with external entities (e.g., users, other systems, organizations)**
- **The flow of data between the system and those external entities**

Some key characteristics of a context diagram are;

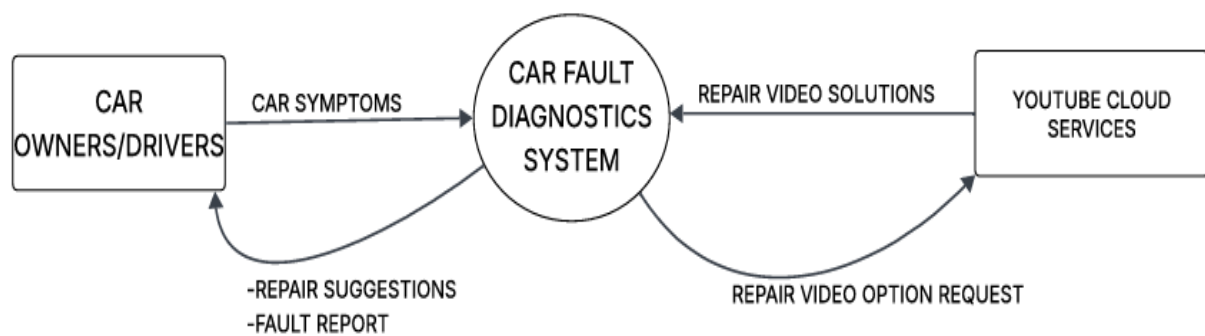
1. **Top-level view:** It does not show internal processes, data stores, or detailed logic.
2. **External entities:** Represented as boxes outside the system boundary.
3. **The system:** Represented as a single process or circle in the center.

4. **Data flows:** Arrows showing what data is sent or received between the system and external entities.

Purpose of the context diagram:

- To establish **boundaries** of the system
- To make sure the system align with its Scope
- To provide a **simple, easy-to-understand** visual overview of system interactions

Below is the context diagram which illustrates the system boundaries and interactions with external entity



Context diagram

Description of the context flow diagram

External Entity 1: Car Owners/Drivers

- These are the users who interact with the system.

Data flows :

- **To System:**
CAR SYMPTOMS — The driver provides information about the car's issues (e.g., noises, warning lights).

- **From System:**
repair suggestions — Video Suggestions for fixing the issue.
Fault report — A diagnostic summary or report generated based on the symptoms.

External Entity 2: YouTube Cloud Services

- This represents the external platform (like YouTube) used to fetch relevant video repair guides.

Data flows:

- **To System:**
repair video solutions — Videos retrieved from YouTube matching the fault.
- **From System:**
repair video option request — The system sends a request to YouTube to retrieve videos related to the identified fault.

IV.2 Use case Diagram

A **Use Case Diagram** is a type of behavioral diagram defined by UML (Unified Modeling Language) that visually represents the interactions between users (called actors) and the system. It illustrates what the system will do and not how it will do it. Each interaction is shown as a "use case," which describes a single function or goal that the system must support.

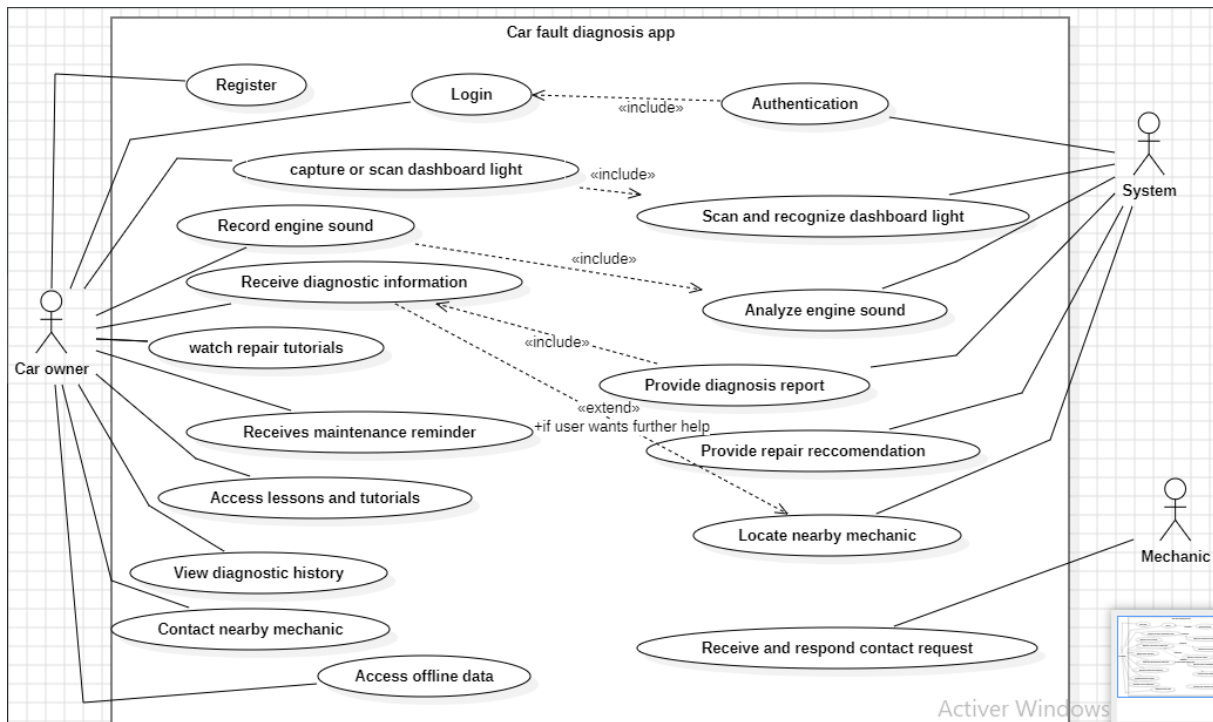
The actors of our system are separated into three categories, the primary actor in this case it is the user who is probably a car owner, secondary actors here the system it self and tertiary actor has little within the system or interacts indirectly with the system in this case it is the mechanic and machine learning models. But in our diagram only the mechanic is present and represented as the tertiary actor.

Various Use Cases and their Description

- **Register and Login:** Allows users to create accounts and access the system securely.

- **Capture Dashboard Light Image:** Enables users to take a picture of their car's dashboard indicator lights using their phone camera.
- **Scan and Recognize Dashboard Light:** The system compares the image to a database and identifies the matching symbol with explanations.
- **Record Engine Sound:** Users can record engine sounds using their phone microphone.
- **Analyze Engine Sound (ML):** The system uses machine learning to classify and identify engine problems from the sound recording.
- **Receive Diagnostic Explanation:** Provides the user with a description of the issue detected, along with its severity and possible causes.
- **Watch Repair Tutorials:** Displays multimedia guides (videos, images) to help users fix identified problems.
- **Log Maintenance Activity:** Users can enter details of maintenance actions taken, forming a vehicle history.
- **Set and Receive Maintenance Reminders:** Sends alerts for scheduled maintenance based on user settings or previous logs.
- **Locate Nearby Mechanic:** Uses GPS to show garages and allow in-app contact with available mechanics.
- **Access Offline Data:** Permits access to previously downloaded tutorials and diagnostic history when the user is offline.
- **Change App Language:** Lets users choose between English, French, or a local language for app navigation.
- **View Diagnostic History:** Allows users to view past diagnostic sessions along with timestamps and outcomes.
- **Access Lessons and Tutorials**

The diagram below illustrates our usecase diagram showing the relationship that exists between the actors and the usecases and the between the various usecases.



IV.3 Class diagram

A Class Diagram is a type of structure diagram in the Unified Modeling Language (UML) that describes the structure of a system by showing its classes, attributes, methods, and the relationships among objects. It is a fundamental building block in object-oriented modeling, used to visualize, specify, construct, and document the artifacts of a software system.

Class diagrams provide a high-level view of a system's architecture and are especially useful during software design and documentation. The key components include:

- **Classes:** Represent entities in the system, containing attributes and methods.
- **Attributes:** Define the data held by a class.
- **Methods:** Represent behaviors or functions a class can perform.
- **Relationships:** Show how classes are associated with each other (e.g., association, inheritance, aggregation).

Our class diagram models a Car Fault Diagnostics System with the following core components:

User and CarOwner:

- The `User` class includes user credentials and methods to register, login, and update profiles.
- The `CarOwner` class extends or associates with `User` and includes language preferences and a

list of owned vehicles.

- Car owners can initiate diagnostics by scanning the dashboard or recording engine sounds.

Vehicle and Maintenance:

- Each ``Vehicle`` object has identifying information and is associated with ``MaintenanceRecord``.
- The ``MaintenanceRecord`` logs services, parts used, and provides reminders and receipts.

Diagnostics:

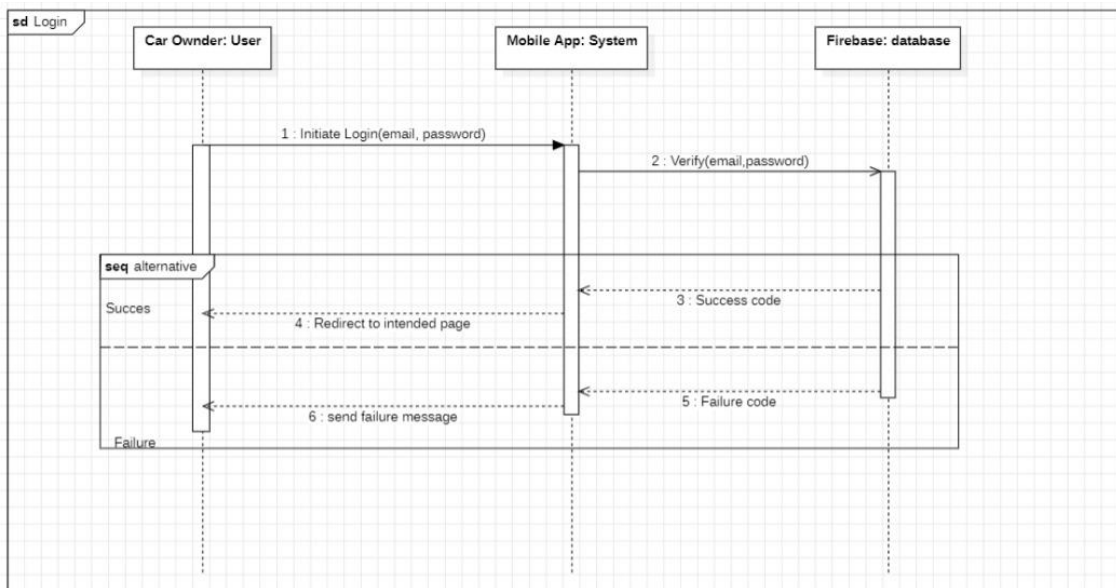
- ``DiagnosticSession`` captures information about a diagnostic attempt, including timestamp and results.
- ``DiagnosticResult`` summarizes findings, including severity and confidence score.
- ``DashboardDiagnosis`` and ``SoundDiagnosis`` inherit or associate with results to analyze dashboard images and audio patterns.

Mechanic and Garage:

- The ``Mechanic`` class stores location, specialization, and certification info, and allows query responses and availability updates.
- The ``Garage`` class provides navigation and distance calculation services.

Notification and Tutorials:

- ``Notification`` handles sending updates to users and logs delivery status.
- ``RepairTutorial`` is associated with ``DiagnosticResult``, offering tutorial content and video playback functionality.

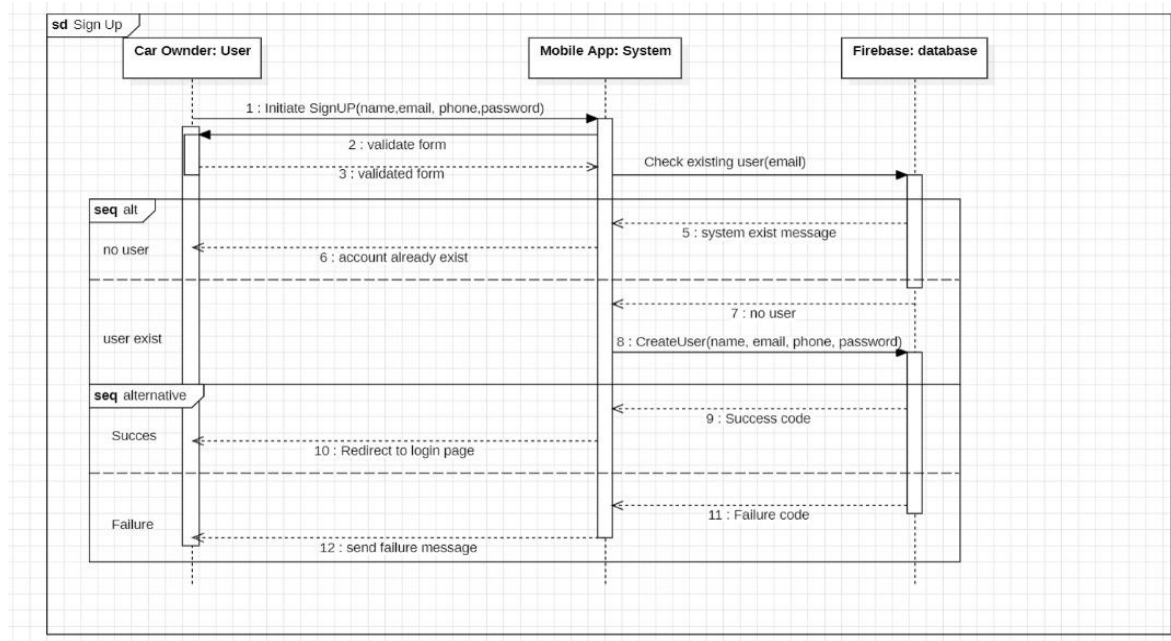


Description

The process to login from the sequence above is

- The user initiates login by going to the login page, filling the login form with their email and password and then clicking the sign in button.
- When this form is submitted the car fault diagnostics system then checks for the availability of these user credentials in the database.
- If the user credentials exist, a success code is sent to the system which then responds to the user by redirecting them to the intended page. Maybe the user is trying to view recommendations they'll be directed to the recommendation page and so on.
- If the user credentials don't exist a failure code is sent to the system which then sends a failure message to the user from which they could correct their credentials or create an account.

2. Create account:

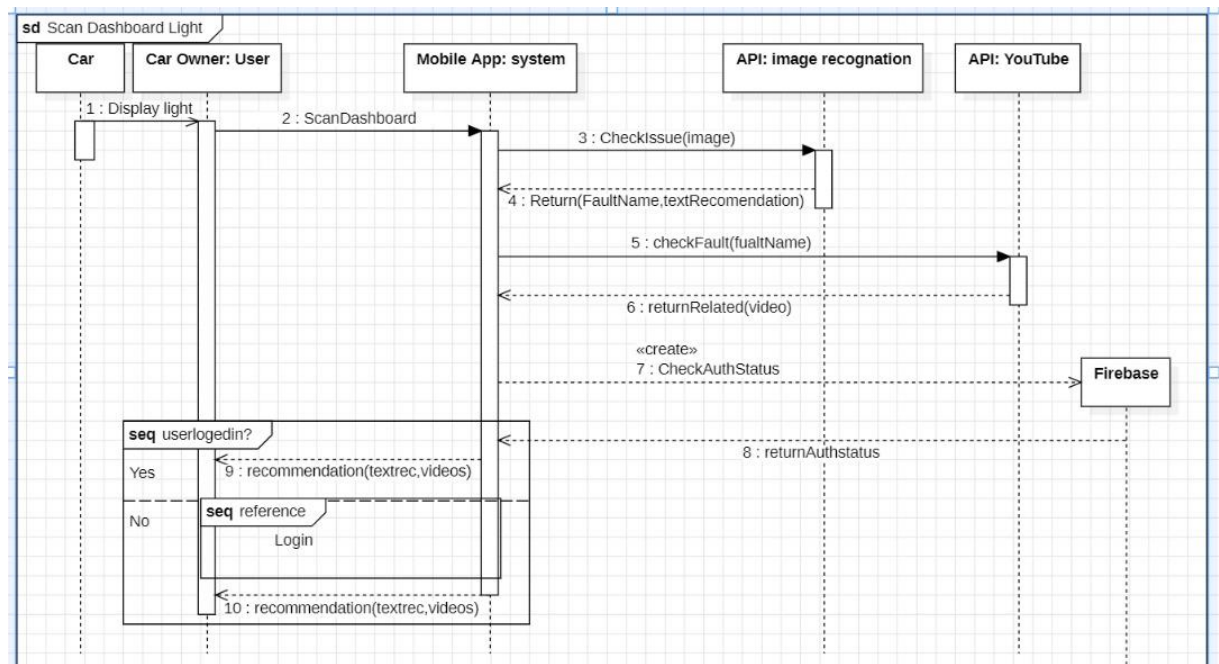


Description:

The process for the user to create an account as depicted by the sequence above is

- The user who is already in the account creation page initiates signup by filling the signup form with their name, email, phone number and password and then clicking the signup button.
- The system ensures that the entered fields follow standards and datatypes and ask the user to validate the for if there are any issues.
- The user then validates the form based on guide lines
- While the system was validating the form, it was also checking if the user name and email already exist in the database.
- If email entered for account creation already exists, the system notifies the user about this from where they can update the form with new credentials or go to the login page.
- If user credentials don't exist, the system creates the account saves it in the database and redirect the user to the login page to login to the account they just created.
- If by any means the account creation failed the user is sent a message that explains the error.

3. Dashboard light scanner

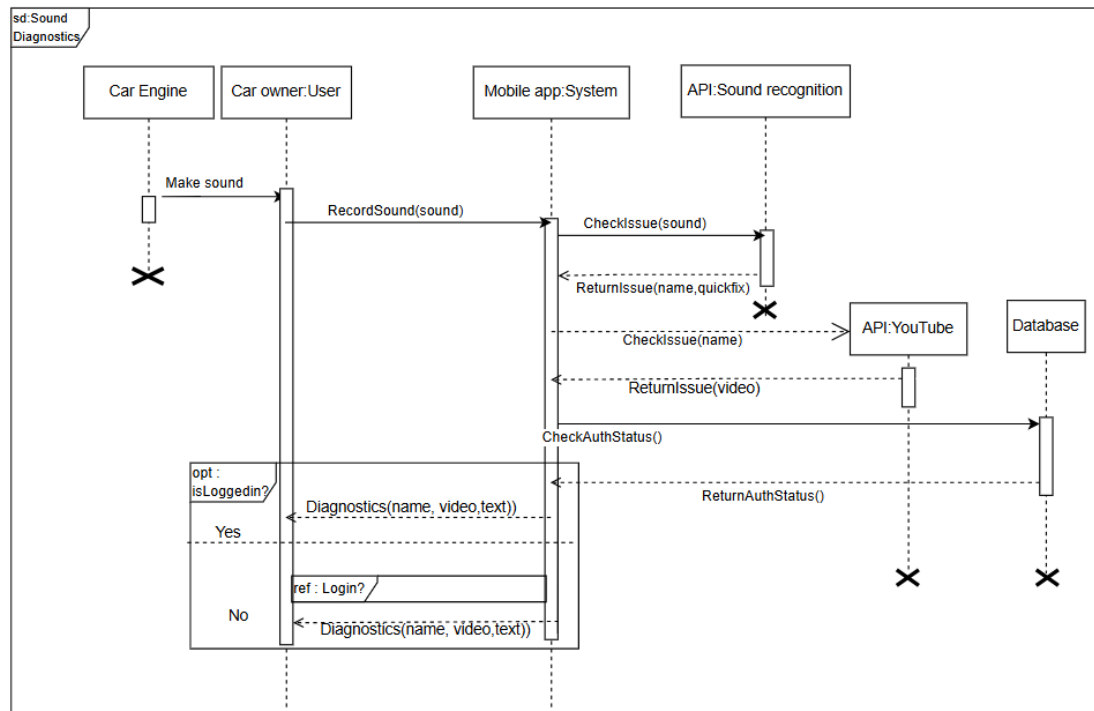


Description:

The process of scanning dashboard lights is as follows

- The car dashboard displays a light (a possible fault)
- The user then uses the App which makes use of their phone camera to scan the light
- The system then takes the scanned image and sends it to the image recognition API (model) that then links this fault with its corresponding name and quick diagnostic recommendation.
- When the system receives this info it then uses this fault name to search the YouTube API for video recommendations
- After the YouTube API returns its response the system then checks if the user is an authorized user i.e if they are logged in and if not, they are asked to login or else the summary of the diagnostics report containing the fault name, text and video quick diagnostics and the approximated fix cost (users can hide either the text or video diagnostics) is displayed to them.

4. Engine sound diagnostics.

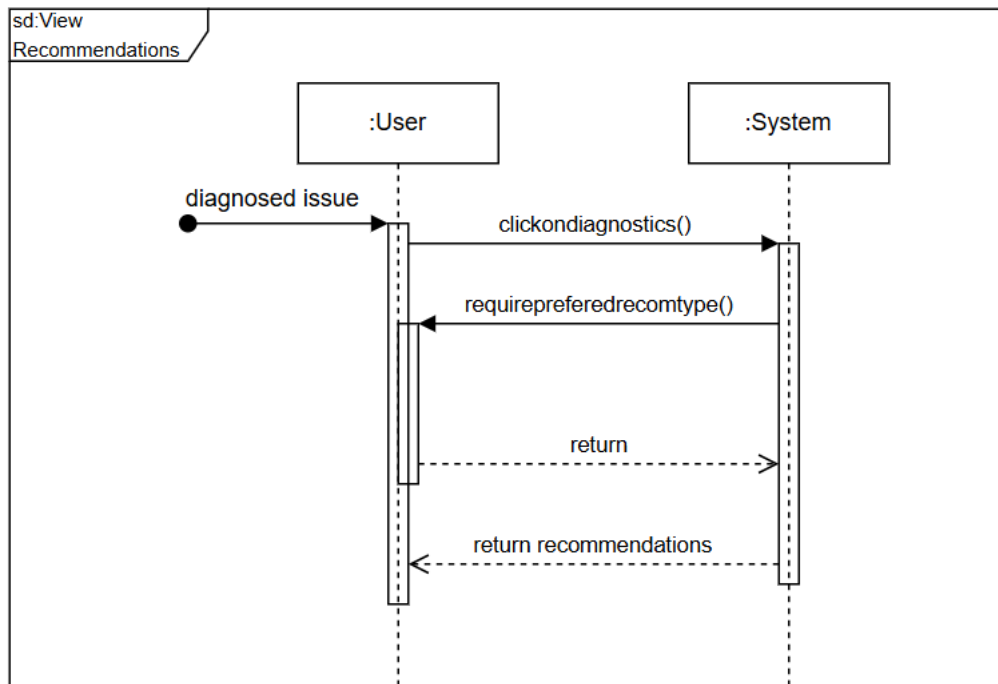


Description:

Similar to the dashboard scanning sequence, the engine sound diagnostics sequence is as follows;

- The car engine makes a sound (a possible fault)
- The user then uses the App which makes use of their phone microphone to record the sound.
- The system then takes the recorded sound and sends it to the sound recognition API (model) that then links this fault with its corresponding name and quick diagnostic recommendation.
- When the system receives this info it then uses this fault name to search the YouTube API for video recommendations
- After the YouTube API returns its response the system then checks if the user is an authorized user i.e if they are logged in and if not, they are asked to login or else the summary of the diagnostics report containing the fault name, text and video quick diagnostics and the approximated fix cost (users can hide either the text or video diagnostics) is displayed to them.

5. View recommendations



Descriptions:

The sequence of steps to view recommendations are as below.

- The user after they most have diagnosed an issue have recommendations returned in a reduced form.
- Users may then click on this to view the full report.
- When clicked the app request the preferred recommendations type which could be text, video or both
- Based on the user response the recommendations are displayed to them fully.

IV.5 Dataflow diagram

A Data Flow Diagram (DFD) is a representation that depicts how data flows within a system, highlighting the processes, data stores, data sources, and destinations. DFDs are used in system analysis and design to model the functional flow of information and clarify how inputs are transformed into outputs.

DFDs typically have four key components:

- **Processes:** Represented by rounded rectangles or circles; show transformations of data.

- **Data Stores:** Represented by open-ended rectangles; show where data is stored.
- **External Entities:** Represented by squares; indicate sources or destinations of data outside the system.
- **Data Flows:** Represented by arrows; indicate the movement of data between components.

Our system dataflow diagram included six main processes which include,

1. 1.0 Authentication

- Car owners provide login or signup details.
- Data is stored in D1: User Profiles, which is accessed by the App Admin for authentication validation.

2. 2.0 Fault Diagnosis (Sound and Dashboard)

- Once authenticated, users initiate a diagnosis.
- OBD-II system performs diagnostics and results are stored in D2: Diagnostic History.
- Data flows to Process 3.0 for further action.

3. 3.0 Repair Recommendation

- Uses diagnostic data to suggest repair actions.
- Incorporates ML Models for intelligent recommendations.
- Results can direct users to tutorials or mechanics.

4. 4.0 Track Maintenance

- Uses D2: Diagnostic History to monitor vehicle maintenance history.

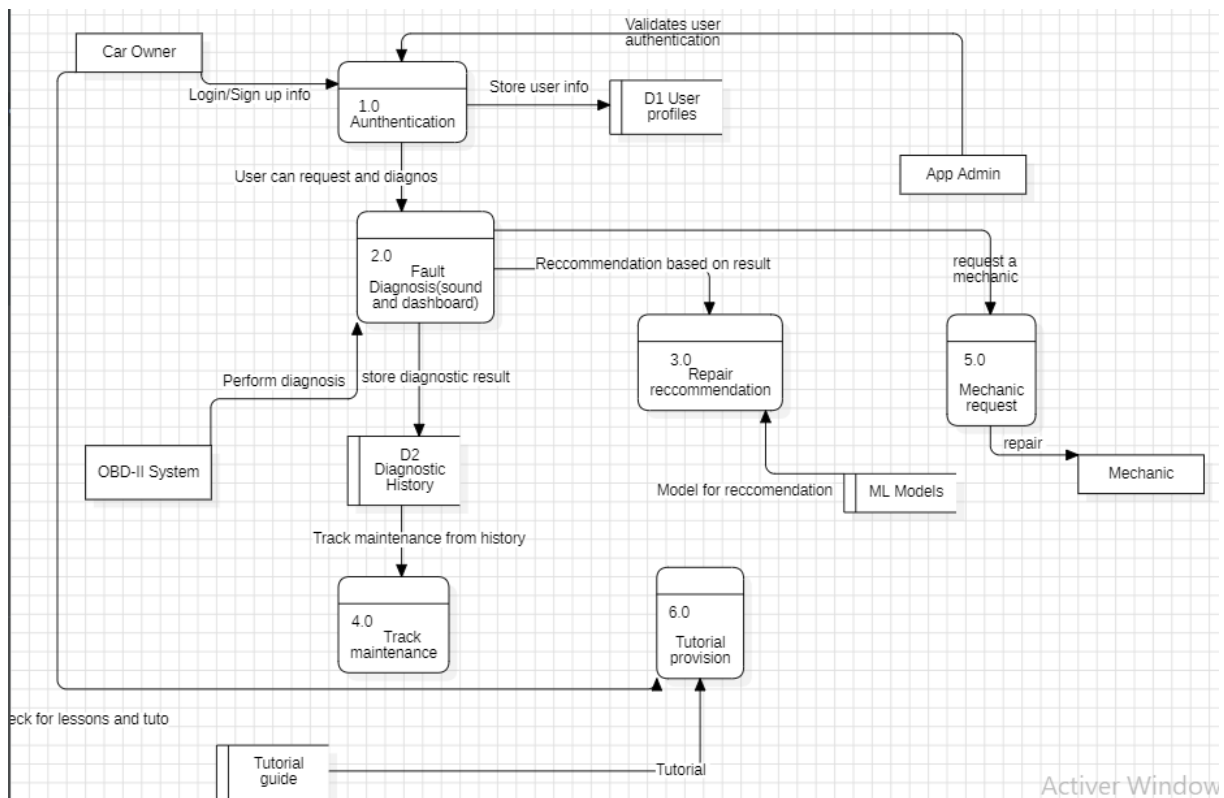
5. 5.0 Mechanic Request

- Allows users to request mechanic intervention.
- Sends a request to the Mechanic based on recommendations or user choice.

6. 6.0 Tutorial Provision

- Based on the diagnostics and repair recommendations, users can access tutorials from the Tutorial Guide.

The diagram below represents our data flow diagram



Dataflow diagram

This DFD outlines a comprehensive flow for diagnosing car issues, recommending repairs, and connecting users with tutorials and mechanics

IV.6 Deployment diagram

Deployment Diagram

A Deployment Diagram is a UML diagram that illustrates the physical architecture of a system, showing how software components are distributed across hardware nodes and how they communicate. It captures:

Nodes: Hardware/software execution environments (e.g., mobile devices, servers).

Artifacts: Deployable units (e.g., APK files, Docker containers).

Communication Paths: Protocols and connections between nodes.

The deployment diagram maps a system's software artifacts to the hardware that will execute them and shows the static view of system software components and hardware in their runtime configuration.

3. Nodes and Components

A. Mobile Device (Android/iOS)

Artifacts:

diagnosis_app.apk (Mobile App executable).

TensorFlow Lite Models: Embedded ML models for offline light/sound analysis.

SQLite Database: Caches fault data and user history.

Hardware:

Camera (for dashboard light scanning).

Microphone (for engine sound recording).

B. Application Server

Artifacts:

API Gateway (Node.js/Express): Routes requests to microservices.

Computer Vision Service (Python/OpenCV): Analyzes dashboard warning lights.

Audio Analysis Service (TensorFlow/PyTorch): Diagnoses engine sounds.

Fault Database (MySQL): Stores fault patterns and repair guides.

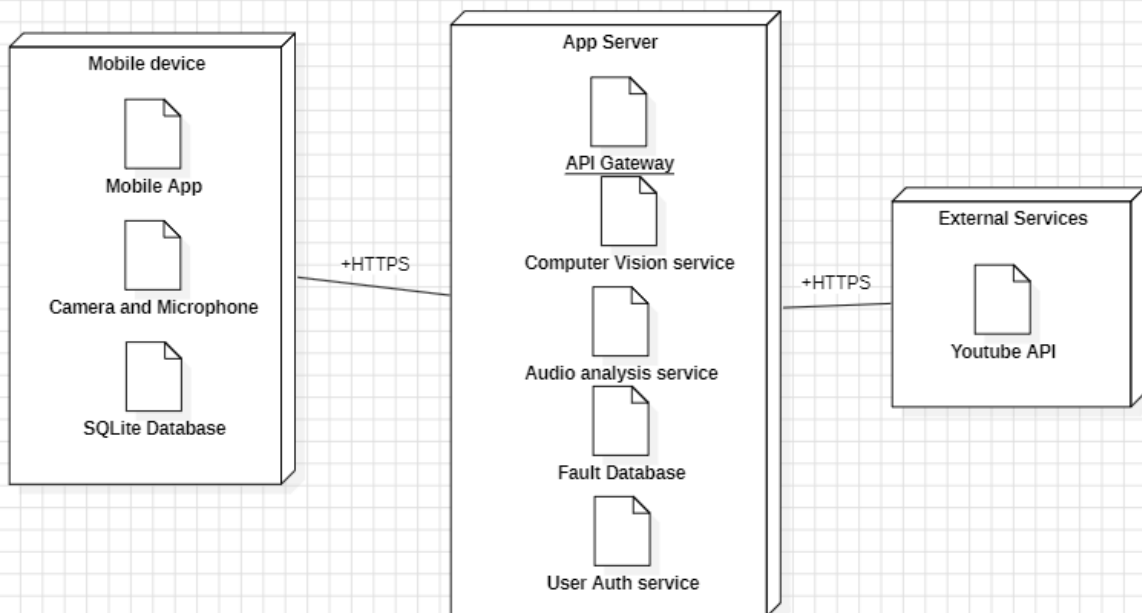
User Auth Service (Firebase): Manages secure logins.

C. External Services

YouTube API: Provides video tutorials (cached for low-bandwidth).

Diagram

Car Fault Diagnosis Deployment diagram



Deployment Diagram

V- DATA DESIGN

The data design, describes how key data is organised in the system. This can easily be visualized from a class diagram but better in an ER (entity relationship diagram) diagram.

From our class diagram, we can see that key data are organised into

User Profiles (ID, name, role)

- Diagnostic Results (date, fault code, explanation)
- ML Models (stored in Firebase or bundled offline)

just to name the main.

VI- COMPONENT DESIGN

Component design is breaking down the system into major components in order to ensure easy development and reusability of components. Here we had to break the system into different components some of which include

- Authentication Component
- Diagnosis part
- Tutorial lessons
- Offline Synchronisation Manager

VII- CONCLUSION

This document describes the complete design of the Car Fault Diagnostics App, including system architecture, data flow, interaction design, and deployment. The design ensures modularity, usability, and extensibility.